Experiment 08 — Load Flow Study Using MATLAB

৪ সদস্যের জন্য লাইন-বাই-লাইন বাংলা কোড গাইড

দায়িত্ব বণ্টন (মোট ~১২ মিনিট)

সদস্য	লাইন	বিষয়	সময়
Member 1	1 – 30	ডেটা ইনপুট ও প্রাথমিক প্রস্তুতি	~৩ মিনিট
Member 2	32 – 73	Ybus গঠন, পরীক্ষা ও প্রদর্শন	~৩ মিনিট
Member 3	75 – 114	Gauss-Seidel iteration ও slack bus power	~৩ মিনিট
Member 4	115 – 186	ফলাফল লেখা (text file) ও দুটি graph	~৩ মিনিট

⚠ গুরুত্বপূর্ণ: কোডে একটি লাইন যোগ করা হয়েছে (লাইন 33)

তোমার পাঠানো কোডে Ybus loop-এর আগে `Y = zeros(no_bus);` লাইনটি ছিল না।

এটা ছাড়া MATLAB প্রথম line-এর ডেটা নেওয়ার পরই "Unrecognized function or variable Y" error দিয়ে থেমে যায় (আমি চালিয়ে নিশ্চিত হয়েছি)। সংযুক্ত .m ফাইলে লাইনটি যোগ করা আছে; নিচের টেবিলে লাল রঙে দেখানো।

লাইন নম্বর সংযুক্ত .m ফাইলের সাথে মিলবে (MATLAB Editor-এর বাম পাশের নম্বর)। সবুজ লাইনগুলো comment।

শুরুর আগে সবার জানা দরকার

Bus-এর ধরন: Slack (|V|, δ জানা; P, Q অজানা) · PV (P, |V| জানা; Q, δ অজানা) · PQ (P, Q জানা; |V|, δ অজানা)।

P = Pg − Pd: bus থেকে network-এ নিট injected power (generation − load)। ধনাত্মক = দিচ্ছে, ঋণাত্মক = নিচ্ছে।

inf: অজানা মান বোঝাতে; line না থাকলে R ও X দুটোতেই inf।

মূল সূত্র: (1) $Q = -\text{Imag}\{V^* \cdot \Sigma YV\}$ (2) $V = (1/Yii)[(P - jQ)/V^* - \Sigma(k \neq i)YV]$ (5) $Sik = Vi[(Vi - Vk)yik]^*$

ফলাফল (4-bus): 10 iteration; V3 = 1.0290∠−5.72°, V4 = 1.0169∠−1.46°, δ2 = 1.65°, Q2 = −0.5278, P1 = 0.2413, Q1 = 0.2558 pu।

Member 1: ডেটা ইনপুট ও প্রাথমিক প্রস্তুতি

লাইন 1 – 30 | সময় ~৩ মিনিট

উপস্থাপনার শুরুতে যা বলবে

আমাদের program Gauss-Seidel পদ্ধতিতে load flow সমাধান করে। প্রথম অংশে program শুরু হয়, কতটি bus তা নেয়, তারপর প্রতিটি bus-এর P, Q, V, δ নেয়। যে মান অজানা, সেখানে inf দিই। এরপর অজানা voltage-কে 1 pu আর angle-কে 0 ধরে complex voltage বানাই, ভুল ইনপুট পরীক্ষা করি, আর কোন bus PV তা চিহ্নিত করি।

লাইন-বাই-লাইন ব্যাখ্যা

লাইন	কোড	বাংলা ব্যাখ্যা
1	`clear; clc; close all;`	clear: workspace-এর সব পুরনো variable মুছে দেয় — আগের run-এর Y, V বা P যেন নতুন হিসাবে ঢুকে ভুল না করে। clc: command window পরিষ্কার করে। close all: আগের খোলা graph বন্ধ করে, যাতে নতুন graph-এর সাথে মিশে না যায়। তাই program সবসময় "পরিষ্কার খাতা" থেকে শুরু হয়।
3	`no_bus=input('Total no of bus is: ')`	input() screen-এ প্রশ্ন দেখিয়ে keyboard থেকে একটি সংখ্যা নেয় এবং no_bus-এ রাখে (manual-এর system-এ 4)। লাইনের শেষে ; নেই, তাই MATLAB "no_bus = 4" আবার দেখায় — ভুল নয়, নিশ্চিত হওয়ার জন্য দেখায়। পরে loop কতবার চলবে আর matrix কত বড় হবে — সব এই no_bus থেকে ঠিক হয়, তাই program যেকোনো সংখ্যক bus-এর জন্য কাজ করে।
5	`P = zeros(1,no_bus); Q = zeros(1,no_bus);`	zeros(1,no_bus) একটি 1×no_bus আকারের শূন্যে ভরা row vector বানায়। P (real power) ও Q (reactive power)-এর জন্য আগে থেকে জায়গা তৈরি (preallocation) — loop-এ মান বসানো দ্রুত ও নিরাপদ হয়। P(2) মানে bus 2-এর P। এক লাইনে দুটি কমান্ড ; দিয়ে আলাদা করা।
6	`V = zeros(1,no_bus); delta = zeros(1,no_bus);`	একইভাবে voltage magnitude V আর angle delta-এর জন্য শূন্য vector। এভাবে প্রতিটি bus-এর চারটি রাশি (P, Q, |V|, δ) চারটি আলাদা vector-এ থাকে — manual-এর hint 4 অনুযায়ী।
8	`for i = 1:no_bus`	for loop — i = 1, 2, ..., no_bus একে একে প্রতিটি bus-এর ডেটা নেওয়া হবে। 1:no_bus মানে 1 থেকে no_bus পর্যন্ত সংখ্যার তালিকা।
9	`fprintf(sprintf('--- Bus %d ---\n', i));`	sprintf() একটি লেখা (string) বানায় যেখানে %d-এর জায়গায় i বসে ("--- Bus 1 ---")। fprintf() সেটা screen-এ প্রিন্ট করে। \n = নতুন লাইন। User যেন বোঝে এখন কোন bus-এর ডেটা চাওয়া হচ্ছে।
10	`P(i) = input(sprintf('P(%d) in pu (type inf if unknown): ', i));`	"P(1) in pu (type inf if unknown): " প্রশ্ন দেখিয়ে উত্তর P(i)-তে রাখা। P = Pg − Pd, অর্থাৎ bus থেকে network-এ নিট injected real power (generation − load)। ধনাত্মক = bus power দিচ্ছে, ঋণাত্মক = নিচ্ছে। Slack bus-এর P অজানা, তাই inf।

লাইন	কোড	বাংলা ব্যাখ্যা
11	`Q(i) = input(sprintf('Q(%d) in pu (type inf if unknown): ', i));`	bus i-এর net reactive power Q = Qg − Qd। Slack ও PV bus-এর Q অজানা → inf। এই inf দেখেই পরে program নিজে PV bus চিনবে।
12	`V(i) = input(sprintf('V(%d) in pu (type inf if unknown): ', i));`	bus i-এর voltage magnitude (pu)। Slack (1.04) ও PV (1.0) bus-এ জানা; PQ bus-এ অজানা → inf।
13	`delta(i) = input(sprintf('delta(%d) in rad (type inf if unknown): ', i));`	bus i-এর voltage angle — radian এককে। Slack bus-এ 0 (reference); বাকিগুলো অজানা → inf।
14	`end`	for loop শেষ। এখন চারটি vector-এ সব bus-এর ডেটা আছে; অজানাগুলো inf।
17	`V(isinf(V)) = 1;`	isinf(V) প্রতিটি ঘরের জন্য true/false দেয় (অসীম হলে true)। V(isinf(V)) = 1 মানে শুধু অজানা ঘরগুলোতে 1 বসাও — একে logical indexing বলে। কারণ: হিসাবে inf ঢুকলে সব উত্তর NaN হয়ে যাবে, তাই একটা যুক্তিসঙ্গত শুরুর অনুমান লাগে। বাস্তবে bus voltage 1 pu-এর কাছাকাছি থাকে (manual-এর hint 1)।
18	`delta(isinf(delta)) = 0;`	অজানা angle-গুলো 0 করা হলো। সব |V| = 1 আর δ = 0 দিয়ে শুরু করাকে "flat start" বলে।
19	`Vc = V .* exp(1j*delta);`	exp(1j*delta) = cos δ + j sin δ (Euler-এর সূত্র)। V .* exp(1j*delta) = |V|∠δ — polar থেকে complex রূপে voltage। .* মানে প্রতিটি bus আলাদাভাবে গুণ। Load flow-এর সব সূত্র complex সংখ্যায়, তাই Vc লাগে। শুরুতে Vc = [1.04, 1, 1, 1]।
21	`% Safety check:`	Comment — নিচে নিরাপত্তা পরীক্ষা। MATLAB comment চালায় না।
22	`for i = 2:no_bus`	Bus 2 থেকে শেষ bus পর্যন্ত loop। Bus 1 বাদ, কারণ slack bus-এর P অজানা হওয়াই স্বাভাবিক।
23	`if isinf(P(i))`	যদি কোনো PV বা PQ bus-এর P ভুল করে inf দেওয়া হয় —
24	`error(['Bus %d: P was entered as inf, but only the slack bus ' ...`	error() সাথে সাথে program থামিয়ে লাল বার্তা দেখায়। [...] দিয়ে কয়েকটি লেখা জোড়া, ... মানে কমান্ড পরের লাইনে চলবে। লজিক: PV ও PQ bus-এ P সবসময় জানা; inf থাকলে equation (2) ভেঙে যাবে। ভুল উত্তর দেওয়ার চেয়ে থেমে কারণ বলা ভালো।
25	`'(bus 1) may have an unknown P. Every PV/PQ bus needs a ' ...`	Error বার্তার দ্বিতীয় অংশ।
26	`'real P value -- re-run and enter the actual number.'], i);`	বার্তার শেষ অংশ; শেষের i দিয়ে %d-এর জায়গায় কোন bus-এ ভুল তা বসে।
27	`end`	if শেষ।
28	`end`	for loop শেষ।

লাইন	কোড	বাংলা ব্যাখ্যা
30	`isPV = isinf(Q);`	isinf(Q) প্রতিটি bus-এর জন্য true/false দেয় — Q অজানা হলে true, অর্থাৎ PV bus। 4-bus-এ isPV = [1 1 0 0]। (Bus 1-ও true, কিন্তু slack বলে iteration loop-এ আসেই না।) কেন এখনই? Iteration-এ Q-এর জায়গায় হিসাব করা মান বসবে, তখন আর inf থাকবে না — পরে দেখলে PV bus-কে PQ ভাবা হতো। তাই মূল ডেটা থেকে একবারেই সংরক্ষণ।

এই অংশের function: কী করে ও কেন

Function	কী করে	কেন ব্যবহার (লজিক)
`clear, clc, close all`	পুরনো variable, screen ও graph মোছে	আগের run-এর মান নতুন হিসাবে মিশে ভুল না করে।
`input()`	keyboard থেকে মান নেয়	ডেটা code-এ লেখা নেই, তাই যেকোনো system দেওয়া যায় (manual-এর নির্দেশ)।
`sprintf()`	%d-এ সংখ্যা বসিয়ে লেখা বানায়	প্রতিটি bus-এর প্রশ্নে সঠিক নম্বর দেখাতে (P(1), P(2) ...)।
`fprintf()`	screen-এ লেখা প্রিন্ট করে	"--- Bus 1 ---" দেখিয়ে user-কে পথ দেখাতে।
`zeros()`	শূন্যে ভরা vector বানায়	P, Q, V, δ-এর জন্য আগে থেকে জায়গা (preallocation)।
`inf, isinf()`	অসীম মান; অসীম কিনা পরীক্ষা	অজানা মান চিহ্নিত করা (manual hint 2), অজানায় শুরুর মান বসানো, PV bus চেনা।
`exp(), 1j`	e^x; কাল্পনিক একক j	|V|∠δ থেকে complex voltage: V·e^(jδ)।
`error()`	program থামিয়ে বার্তা দেখায়	Non-slack bus-এ P = inf দিলে ভুল উত্তরের বদলে কারণ জানায়।
`for … end, if … end`	পুনরাবৃত্তি; শর্তসাপেক্ষ কাজ	প্রতিটি bus-এর জন্য একই প্রশ্ন; ভুল থাকলে তবেই error।

লজিক: "কেন এভাবে?"

- অজানা মান inf কেন? — একটি সংখ্যাতেই "অজানা" বোঝানো যায়, আর isinf() দিয়ে program নিজে bus type চিনে নেয়।
- অজানা V = 1, δ = 0 কেন (flat start)? — Iteration শুরু করতে একটা অনুমান লাগে; বাস্তবে bus voltage 1 pu-এর কাছে ও angle ছোট থাকে, তাই দ্রুত converge করে। inf রেখে দিলে হিসাব NaN হয়ে যেত।
- V(isinf(V)) = 1 — logical indexing: loop ছাড়াই শুধু শর্ত মেলা ঘরগুলো বদলানো যায়।
- isPV কেন এখনই? — পরে Q-এর জায়গায় হিসাব করা মান বসবে, তখন inf থাকবে না; তাই মূল ডেটা থেকেই একবারে PV bus চিহ্নিত।
- P = Pg − Pd — bus থেকে network-এ নিট power। ধনাত্মক = দিচ্ছে (bus 2: +0.5), ঋণাত্মক = নিচ্ছে (bus 3: −1.0)।

সম্ভাব্য ভাইভা প্রশ্ন

প্রশ্ন 1: Slack bus-এর P ও Q কেন অজানা?

উত্তর: Loss আগে জানা যায় না। Slack bus মোট load + loss আর বাকি generation-এর পার্থক্য সরবরাহ করে, তাই তার P, Q শেষে হিসাব হয়।

প্রশ্ন 2: PV আর PQ bus-এর পার্থক্য?

উত্তর: PV-তে P ও |V| জানা, Q ও δ অজানা। PQ-তে P ও Q জানা, |V| ও δ অজানা।

প্রশ্ন 3: no_bus লাইনে ; নেই কেন?

উত্তর: ; না দিলে MATLAB মানটা দেখায় — "no_bus = 4"। শুধু নিশ্চিত হওয়ার জন্য; ফলাফলে প্রভাব নেই।

প্রশ্ন 4: Vc কেন দরকার?

উত্তর: সব load flow সূত্র complex সংখ্যায়; |V| ও δ আলাদা রাখলে সরাসরি গুণ-ভাগ করা যায় না।

Member 2: Ybus গঠন, পরীক্ষা ও প্রদর্শন

লাইন 32 – 73 | সময় ~৩ মিনিট

উপস্থাপনার শুরুতে যা বলবে

দ্বিতীয় অংশে line ডেটা থেকে bus admittance matrix বা Ybus তৈরি হয়। প্রতিটি bus-জোড়ার R ও X নিয়ে admittance y = 1/(R + jX) বের করি; off-diagonal-এ −y আর diagonal-এ +y যোগ করি। Line না থাকলে inf দিলে admittance 0 হয়। তারপর কোনো bus বিচ্ছিন্ন কিনা পরীক্ষা করি, Ybus সুন্দরভাবে দেখাই, আর iteration-এর জন্য counter ও খালি array তৈরি করি।

লাইন-বাই-লাইন ব্যাখ্যা

লাইন	কোড	বাংলা ব্যাখ্যা
32	`%% YBus formation`	Section শুরু (%%) — Ybus গঠন (manual-এর step 5)।
33	`Y = zeros(no_bus);`	zeros(no_bus) একটি no_bus × no_bus শূন্য matrix বানায় (4×4)। খুব গুরুত্বপূর্ণ: নিচের লাইনে Y(i,k) = Y(i,k) − y লেখা — ডান দিকের Y(i,k) আগে থেকে না থাকলে MATLAB "Unrecognized function or variable Y" error দেয়। তাই সব ঘর 0 দিয়ে শুরু করে ধাপে ধাপে যোগ করা হয়। (তোমার পাঠানো কোডে এই লাইন বাদ পড়েছিল — এটা ছাড়া কোড চলবে না।)
34	`for i = 1:no_bus`	বাইরের loop — i = 1 থেকে no_bus।

লাইন	কোড	বাংলা ব্যাখ্যা
35	`for k = i+1:no_bus`	ভেতরের loop — k = i+1 থেকে no_bus। এতে শুধু 1-2, 1-3, 1-4, 2-3, 2-4, 3-4 জোড়া আসে; 2-1 বা 1-1 আসে না, কারণ line 1-2 আর 2-1 একই। দুটি nested loop দিয়ে সবচেয়ে কম বার — manual-এর নির্দেশ অনুযায়ী।
36	`fprintf(sprintf('--- Line %d-%d (type inf,inf if no line) ---\n', i, k));`	কোন line-এর ডেটা চাওয়া হচ্ছে দেখায়। Line না থাকলে R ও X দুটোতেই inf দিতে হবে (যেমন line 1-4)।
37	`r = input(sprintf('R(%d,%d) in pu: ', i, k));`	Line i-k-এর resistance R (pu) নেওয়া হয় r-এ।
38	`x = input(sprintf('X(%d,%d) in pu: ', i, k));`	Line i-k-এর reactance X (pu) নেওয়া হয় x-এ।

লাইন	কোড	বাংলা ব্যাখ্যা
39	`y = 1/(r + 1j*x);`	Series admittance y = 1/Z = 1/(R + jX); 1j = কাল্পনিক একক j। যেমন line 1-2: 1/(0.05 + j0.15) = 2 − j6। Line না থাকলে r = x = inf → 1/inf = 0, তাই Ybus-এ কিছু যোগ হয় না — এভাবে "line নেই" বোঝানো হয়।
40	`Y(i,k) = Y(i,k) - y; Y(k,i) = Y(i,k);`	Off-diagonal নিয়ম: Yik = −yik। Y(i,k) থেকে y বিয়োগ, তারপর Y(k,i) = Y(i,k) — matrix symmetric (line দুই দিক থেকে একই)।
41	`Y(i,i) = Y(i,i) + y; Y(k,k) = Y(k,k) + y;`	Diagonal নিয়ম: Yii = ওই bus-এ যুক্ত সব line-এর admittance-এর যোগফল। Line i-k দুই bus-এই যুক্ত, তাই Y(i,i) ও Y(k,k) দুটোতেই +y। সব line শেষে Y11 = y12 + y13 = 3 − j9। লজিক (nodal analysis): Ii = Σ yik(Vi − Vk) → Vi-এর সহগ Σyik, Vk-এর সহগ −yik।

লাইন	কোড	বাংলা ব্যাখ্যা
42	`end`	ভেতরের loop শেষ।
43	`end`	বাইরের loop শেষ — Ybus তৈরি সম্পূর্ণ।
45	`% Safety check:1 line connect`	Comment — নিরাপত্তা পরীক্ষা: প্রতিটি bus কমপক্ষে একটি line-এ যুক্ত থাকতে হবে।
46	`for i = 1:no_bus`	সব bus-এর জন্য loop।
47	`if Y(i,i) == 0`	Y(i,i) == 0 মানে bus i-এর সাথে কোনো line নেই।
48	`error(['Bus %d has no connecting line (Y(%d,%d)=0), so its ' ...`	তাহলে error দিয়ে থামাও — কারণ equation (2)-তে 1/Y(i,i) আছে; Y(i,i) = 0 হলে শূন্য দিয়ে ভাগ হবে।
49	`'voltage equation divides by zero. Check the R/X you ' ...`	Error বার্তার পরের অংশ।
50	`'entered for every line touching this bus.'], i, i, i);`	বার্তার শেষ অংশ; তিনটি i দিয়ে তিনটি %d পূরণ।
51	`end`	if শেষ।
52	`end`	for loop শেষ।
54	`fprintf('\n==================== BUS ADMITTANCE MATRIX (YBus) in pu ====================\n');`	Ybus টেবিলের শিরোনাম প্রিন্ট; শুরুর \n একটি ফাঁকা লাইন দেয়।

লাইন	কোড	বাংলা ব্যাখ্যা
55	`fprintf('%8s', '');`	%8s = 8 ঘর চওড়া জায়গায় একটি লেখা — এখানে খালি, তাই 8টি space। বাম দিকের "Bus 1" লেবেলের জায়গা ফাঁকা রাখা হলো যাতে column শিরোনাম সংখ্যার ঠিক উপরে বসে।
56	`for k = 1:no_bus`	প্রতিটি column শিরোনামের জন্য loop।
57	`fprintf('%22s', sprintf('Bus %d', k));`	sprintf দিয়ে "Bus 1", "Bus 2" ... বানিয়ে %22s দিয়ে 22 ঘর জায়গায় ডানদিকে সাজিয়ে প্রিন্ট — নিচের সংখ্যার সাথে column মেলে।
58	`end`	শিরোনাম loop শেষ।
59	`fprintf('\n');`	নতুন লাইনে যাওয়া।
60	`for i = 1:no_bus`	Ybus-এর প্রতিটি row-এর জন্য loop।
61	`fprintf('  Bus %d ', i);`	row-এর শুরুতে " Bus 1 " লেবেল।
62	`for k = 1:no_bus`	ওই row-এর প্রতিটি column-এর জন্য loop।

লাইন	কোড	বাংলা ব্যাখ্যা
63	`fprintf('   %9.4f %+9.4fj', real(Y(i,k)), imag(Y(i,k)));`	real() = real অংশ, imag() = imaginary অংশ। %9.4f = 9 ঘরে দশমিকের পর 4 অঙ্ক; %+9.4f = চিহ্নসহ (+/−), তারপর j। complex সংখ্যা fprintf-এ সরাসরি লেখা যায় না, তাই দুই ভাগে লেখা।
64	`end`	column loop শেষ।
65	`fprintf('\n');`	row শেষে নতুন লাইন।
66	`end`	row loop শেষ — পুরো matrix প্রিন্ট।
67	`fprintf('================================================================================\n');`	টেবিলের নিচে দাগ।
68	`Ybus = Y`	শেষে ; নেই, তাই MATLAB নিজের format-এ "Ybus = ..." দেখায় (i দিয়ে complex) — দ্বিতীয়বার যাচাই। Y14 = 0 দেখা যাবে কারণ bus 1 ও 4-এর মাঝে line নেই।
70	`%% iteration counter`	Section — iteration counter (manual-এর step 6)।

লাইন	কোড	বাংলা ব্যাখ্যা
71	`iter = 0;`	iter = 0 — এখনো কোনো iteration হয়নি; loop-এ প্রতিবার 1 বাড়বে। শেষে এটাই "Result found at iteration number"।
72	`del_hist = [];`	খালি array — প্রতিটি iteration-এর সবচেয়ে বড় পরিবর্তন (del) জমা হবে → convergence graph।
73	`delta_hist = [];`	খালি array — প্রতিটি iteration শেষে সব bus-এর angle জমা হবে (row = iteration, column = bus) → angle graph।

এই অংশের function: কী করে ও কেন

Function	কী করে	কেন ব্যবহার (লজিক)
`zeros(no_bus)`	n×n শূন্য matrix	Ybus-এ ধাপে ধাপে যোগ করতে শুরুতে সব 0 থাকতে হয় (নাহলে Y-এর error)।
`nested for`	দুটি loop একটির ভেতরে	প্রতিটি bus-জোড়া (i < k) একবারই — সবচেয়ে কম loop (manual-এর নির্দেশ)।
`1/(r + 1j*x)`	impedance থেকে admittance	Ybus admittance দিয়ে গঠিত। inf দিলে 1/inf = 0 → line নেই।
`error()`	থামিয়ে বার্তা	Y(i,i) = 0 হলে পরে শূন্য দিয়ে ভাগ হতো।
`real(), imag()`	complex-এর দুই অংশ	fprintf complex সরাসরি লিখতে পারে না, তাই ভেঙে লেখা।
`fprintf format`	%8s, %22s, %9.4f, %+9.4f	Column সোজা রাখতে; + দিয়ে সবসময় চিহ্ন দেখাতে।
`[]`	খালি array	del_hist ও delta_hist-এ iteration চলতে চলতে মান যোগ হবে।

লজিক: "কেন এভাবে?"

- Yik = −yik, Yii = Σyik কেন? — Nodal analysis: Ii = Σ yik(Vi − Vk)। Vi-এর সহগ Σyik (diagonal), Vk-এর সহগ −yik (off-diagonal)।
- k = i+1 থেকে কেন? — line 1-2 আর 2-1 একই; আর 1-1 বলে কোনো line নেই। তাই শুধু উপরের ত্রিভুজ; Y(k,i) = Y(i,k) দিয়ে নিচের অংশ পূরণ (symmetric)।
- Line i-k-এর y কেন দুই diagonal-এই যোগ? — line টা দুই bus-এই যুক্ত।
- Y = zeros(no_bus) কেন অবশ্যই লাগবে? — Y(i,k) = Y(i,k) − y লাইনের ডান দিকে Y আগে থেকে থাকতে হবে। পাঠানো কোডে এই লাইন ছিল না, তাই "Unrecognized function or variable Y" error দিত; আমরা যোগ করেছি।
- Ybus দুইবার দেখানো কেন? — প্রথমটা পড়তে সহজ টেবিল, দ্বিতীয়টা (Ybus = Y) MATLAB-এর নিজস্ব format, যাচাইয়ের জন্য।

সম্ভাব্য ভাইভা প্রশ্ন

প্রশ্ন 1: Ybus কী?

উত্তর: Bus admittance matrix — কোন bus কোন bus-এর সাথে কতটা admittance দিয়ে যুক্ত তা বোঝায়; load flow-এর সব সূত্রে ব্যবহৃত হয়।

প্রশ্ন 2: Y14 = 0 কেন?

উত্তর: Bus 1 ও 4-এর মাঝে line নেই (R, X = inf দেওয়া হয়েছে)।

প্রশ্ন 3: Ybus sparse ও symmetric কেন?

উত্তর: প্রতিটি bus অল্প কয়েকটি bus-এর সাথে যুক্ত, তাই অনেক শূন্য; Yik = Yki কারণ line দুই দিকে একই।

প্রশ্ন 4: Y11 হিসাব করো।

উত্তর: y12 + y13 = (2 − j6) + (1 − j3) = 3 − j9।

Member 3: Gauss-Seidel iteration ও slack bus power

লাইন 75 – 114 | সময় ~৩ মিনিট

উপস্থাপনার শুরুতে যা বলবে

তৃতীয় অংশটাই program-এর মূল। while loop যতক্ষণ সবচেয়ে বড় পরিবর্তন del ≥ 0.00017, ততক্ষণ চলে। প্রতি iteration-এ bus 2 থেকে শেষ bus পর্যন্ত যাই। PV bus হলে আগে equation (1) দিয়ে Q, তারপর equation (2) দিয়ে নতুন angle, |V| স্থির থাকে। PQ bus হলে equation (2) দিয়ে magnitude ও angle দুটোই। নতুন মান সাথে সাথে ব্যবহার করি — এটাই Gauss-Seidel। শেষে slack bus-এর P ও Q বের করি। আমাদের system 10 iteration-এ converge করেছে।

লাইন-বাই-লাইন ব্যাখ্যা

লাইন	কোড	বাংলা ব্যাখ্যা
75	`%% Gauss-Seidel sweep`	Section — Gauss-Seidel iteration (manual-এর step 7-9)। Bus 1 (slack) কখনো বদলানো হয় না, কারণ তার V ও δ জানা।
76	`del = 1;`	del = 1 — while-এর শর্ত প্রথমবার সত্য করার জন্য একটা বড় মান; নাহলে loop একবারও চলত না।
77	`while del >= 0.000170`	while loop — যতক্ষণ del ≥ 0.00017 (মান এখনো উল্লেখযোগ্যভাবে বদলাচ্ছে), ততক্ষণ চলবে। del এর চেয়ে ছোট হলে উত্তর স্থির ধরে থামে। কতবার লাগবে আগে জানা নেই, তাই for নয়, while। (Manual-এ 0.0001 বলা; এখানে 0.00017 ।)
78	`iter = iter + 1;`	প্রতি নতুন iteration-এ counter 1 বাড়ে।
79	`del = 0;`	এই iteration-এর শুরুতে del = 0; প্রতিটি পরিবর্তনের সাথে তুলনা করে সবচেয়ে বড়টা থাকবে (step 7a)।
80	`for i = 2:no_bus`	Bus 2 থেকে শেষ bus পর্যন্ত loop — slack বাদ।
81	`sum_term = Y(i,:)*Vc.' - Y(i,i)*Vc(i);`	Σ(k≠i) YikVk বের করা। Y(i,:) = Ybus-এর i-তম row; Vc.' = voltage column (.' = conjugate ছাড়া transpose; শুধু ' দিলে conjugate হয়ে ভুল হতো)। row × column = ΣYikVk; এর থেকে নিজের পদ Y(i,i)Vc(i) বাদ দিলে k ≠ i-এর যোগফল — equation (2)-এ এটাই লাগে।
83	`if isPV(i)`	isPV(i) সত্য হলে bus i PV bus — নিচের PV অংশ চলবে।
84	`Q_new = -imag( conj(Vc(i)) * (Y(i,:)*Vc.') );`	Equation (1): Q = −Imag{Vi* · ΣYikVk}। conj() = Vi*, imag() = imaginary অংশ। PV bus-এ Q অজানা কিন্তু equation (2)-এ লাগে, তাই সর্বশেষ voltage দিয়ে আগে Q বের হয়। প্রথম iteration-এ Q2 = −0.24।
85	`V_new = ( (P(i) - 1j*Q_new)/conj(Vc(i)) - sum_term ) / Y(i,i);`	Equation (2): Vi = (1/Yii)[(Pi − jQi)/Vi* − Σ(k≠i)YikVk] — সদ্য পাওয়া Q_new দিয়ে। এটাই Gauss-Seidel-এর মূল সূত্র, Si* = Vi*·ΣYikVk থেকে Vi আলাদা করে পাওয়া।
86	`delta_new = angle(V_new);`	angle() = complex সংখ্যার কোণ (radian)। PV bus-এর |V| নির্দিষ্ট (1.0 pu), তাই V_new-এর শুধু angle নেওয়া হয়; magnitude ফেলে দেওয়া হয়।

লাইন	কোড	বাংলা ব্যাখ্যা
87	`del = max([del, abs(delta_new - delta(i))]);`	abs() = পরম মান। পুরনো ও নতুন angle-এর পার্থক্য del-এর সাথে তুলনা করে বড়টা রাখা (max)।
88	`delta(i) = delta_new;`	নতুন angle বসানো (step 7b)। V(i) বদলানো হয়নি — specified মানই থাকে।
89	`Q(i) = Q_new;`	হিসাব করা Q সংরক্ষণ — output-এ Q2 = −0.5278 এখান থেকেই আসে।
90	`else % (PQ bus)`	else — PV না হলে এটা PQ (load) bus। % এর পরের লেখা comment।
91	`V_new = ( (P(i) - 1j*Q(i))/conj(Vc(i)) - sum_term ) / Y(i,i);`	Equation (2) — এখানে Q(i) input থেকে জানা, তাই সরাসরি ব্যবহার।
92	`V_mag_new = abs(V_new);`	abs(V_new) = নতুন voltage magnitude।
93	`delta_new = angle(V_new);`	angle(V_new) = নতুন voltage angle (radian)।
94	`del = max([del, abs(V_mag_new - V(i)), abs(delta_new - delta(i))]);`	magnitude-এর পরিবর্তন ও angle-এর পরিবর্তন — দুটোই del-এর সাথে তুলনা করে সবচেয়ে বড়টা রাখা।
95	`V(i) = V_mag_new;`	নতুন magnitude বসানো।
96	`delta(i) = delta_new;`	নতুন angle বসানো।
97	`end`	if-else শেষ।
98	`Vc(i) = V(i) * exp(1j*delta(i));`	নতুন |V| ও δ থেকে complex voltage বানিয়ে সাথে সাথেই Vc(i)-তে বসানো। ফলে পরের bus-এর sum_term-এ এই নতুন মান ব্যবহার হয় — এটাই Gauss-Seidel (Gauss/Jacobi-তে পুরো iteration শেষে একসাথে update হয়, তাই ধীর)।
99	`if ~isfinite(V(i)) || ~isfinite(delta(i))`	isfinite() দেখে মানটি সাধারণ সংখ্যা কিনা (inf/NaN নয়)। ~ = NOT, || = OR। V বা δ অসীম/NaN হলে —
100	`error(['Divergence at bus %d, iteration %d (value became ' ...`	error দিয়ে থামাও — হিসাব diverge করেছে। অর্থহীন উত্তর বা অসীম loop এড়াতে।
101	`'NaN/Inf). Double check the R/X and P/Q/V/delta you ' ...`	Error বার্তার পরের অংশ।
102	`'entered -- a stray inf in the wrong field is the ' ...`	Error বার্তার পরের অংশ।
103	`'usual cause.'], i, iter);`	শেষ অংশ; i ও iter দিয়ে কোন bus ও কোন iteration-এ সমস্যা তা দেখায়।
104	`end`	if শেষ।
105	`end`	Bus loop শেষ — একটি পূর্ণ iteration সম্পন্ন।
106	`% iteration's result save`	Comment — এই iteration-এর ফল graph-এর জন্য সংরক্ষণ।
107	`del_hist(iter) = del;`	del_hist-এর iter-তম ঘরে এই iteration-এর del — convergence graph-এর y-মান। Array নিজে থেকে বড় হয়।
108	`delta_hist(iter,:) = delta*180/pi;`	delta radian-এ; ×180/pi করে ডিগ্রিতে, delta_hist-এর iter-তম row-এ সব bus-এর angle — angle graph-এর ডেটা।
109	`end`	while loop শেষ। MATLAB আবার শর্ত (del ≥ 0.00017) দেখে; মিথ্যা হলে বের হয়। 4-bus system-এ 10টি iteration লাগে।

লাইন	কোড	বাংলা ব্যাখ্যা
111	`%% slack bus eqn`	Section — slack bus-এর power (manual-এর step 10, equation 3 ও 4)।
112	`S1 = Vc(1) * conj( Y(1,:)*Vc.' );`	S1 = V1 × conj(ΣY1kVk) = V1 × I1* = bus 1-এর injected complex power S = P + jQ। (Manual-এ S* = V*·I রূপে; এখানে সরাসরি S = V·I* — একই জিনিস।)
113	`P(1) = real(S1);`	real(S1) = slack bus-এর real power P1 = 0.2413 pu। Manual-এ abs লেখা থাকলেও real power হলো real অংশ; abs দিলে apparent power আসত।
114	`Q(1) = imag(S1);`	imag(S1) = Q1 = 0.2558 pu। S = V·I* হওয়ায় imag সরাসরি Q দেয়, − চিহ্ন লাগে না (manual-এর S* রূপে −Imag লাগত)।

এই অংশের function: কী করে ও কেন

Function	কী করে	কেন ব্যবহার (লজিক)
`while … end`	শর্ত সত্য থাকা পর্যন্ত চলে	কতবার লাগবে আগে জানা নেই — del ছোট না হওয়া পর্যন্ত।
`Y(i,:)*Vc.'`	row × column = ΣYikVk	পুরো যোগফল এক লাইনে, loop ছাড়া। .' = conjugate ছাড়া transpose।
`conj()`	complex conjugate	সূত্রে Vi* ও I* লাগে।
`imag(), real()`	imaginary ও real অংশ	Q = −Imag{…}; P1 = real(S1), Q1 = imag(S1)।
`angle()`	কোণ (radian)	PV bus-এ শুধু নতুন δ নেওয়া; PQ-তেও δ।
`abs()`	magnitude / পরম মান	|V| বের করা; পরিবর্তনের মাপ ধনাত্মক রাখা।
`max()`	সবচেয়ে বড় মান	সব variable-এর পরিবর্তনের মধ্যে বড়টা del — সবচেয়ে ধীরটাও স্থির হলে তবেই থামবে।
`isfinite()`	inf/NaN কিনা	Divergence ধরা।
`if isPV(i) … else`	শর্তসাপেক্ষ	PV আর PQ bus-এর হিসাব আলাদা।

লজিক: "কেন এভাবে?"

- del = 1 দিয়ে শুরু কেন? — while-এর শর্ত প্রথমবার সত্য না হলে loop একবারও চলত না।
- sum_term থেকে Y(i,i)Vc(i) কেন বাদ? — equation (2)-এ Σ(k≠i) লাগে।
- PV bus-এ আগে Q কেন? — equation (2)-এ Q লাগে কিন্তু PV bus-এ Q অজানা; তাই সর্বশেষ voltage দিয়ে আগে Q।
- PV bus-এ শুধু angle কেন? — |V| generator নির্দিষ্ট রাখে (1.0 pu); equation (2)-এর magnitude ফেলে দিই।
- Vc(i) সাথে সাথে update কেন? — Gauss-Seidel: নতুন মান পরের bus-এর হিসাবে ব্যবহার হয়, তাই Gauss (Jacobi)-এর চেয়ে দ্রুত।
- Q1 = imag(S1), manual-এ −Imag কেন? — এখানে S1 = V1·I1* (সরাসরি S), manual-এ S* = V*·I; দুটো একই ফল।
- P1 = real, manual-এর abs নয় কেন? — abs দিলে apparent power |S| আসে; real power হলো real অংশ।

সম্ভাব্য ভাইভা প্রশ্ন

প্রশ্ন 1: Gauss আর Gauss-Seidel-এর পার্থক্য?

উত্তর: Gauss-এ পুরো iteration শেষে একসাথে update; Gauss-Seidel-এ নতুন মান সাথে সাথেই পরের bus-এ ব্যবহার — তাই দ্রুত।

প্রশ্ন 2: প্রথম iteration-এ Q2 কত? কীভাবে?

উত্তর: ΣY2kVk = −0.08 + j0.24; Q2 = −Imag{1 × (−0.08 + j0.24)} = −0.24 pu।

প্রশ্ন 3: প্রথম iteration-এ del = 0.0847 কেন?

উত্তর: Bus 3-এর angle 0 থেকে −4.8529° = 0.0847 rad হয়েছে — সব পরিবর্তনের মধ্যে এটাই সবচেয়ে বড়।

প্রশ্ন 4: Convergence criterion কী?

উত্তর: এক iteration-এ সবচেয়ে বড় পরিবর্তন del < 0.00017 হলে থামে।

প্রশ্ন 5: Slack bus কত power দিচ্ছে?

উত্তর: P1 = 0.2413 pu, Q1 = 0.2558 pu।

Member 4: ফলাফল লেখা (text file) ও দুটি graph

লাইন 115 – 186 | সময় ~৩ মিনিট

উপস্থাপনার শুরুতে যা বলবে

চতুর্থ অংশে সব ফলাফল load_flow_result.txt ফাইলে লেখা হয়: iteration সংখ্যা, Ybus, প্রতিটি bus-এর V, angle, P, Q, প্রতিটি line-এর দুই দিকের power flow (equation 5), আর iteration history। তারপর দুটি graph আঁকি — convergence graph, যেখানে del log scale-এ কমে tolerance রেখার নিচে নামে; আর bus voltage angle graph, যেখানে প্রতিটি bus-এর angle কীভাবে স্থির মানে পৌঁছায় তা দেখা যায়।

লাইন-বাই-লাইন ব্যাখ্যা

লাইন	কোড	বাংলা ব্যাখ্যা
115	`fid = fopen('load_flow_result.txt', 'w');`	fopen() "load_flow_result.txt" ফাইল লেখার জন্য খোলে ('w' = write, আগের লেখা মুছে নতুন) এবং file id fid-এ দেয়। এরপর fprintf(fid, ...) দিলে লেখা ফাইলে যায় — manual-এর নির্দেশ (fprintf দিয়ে text file-এ ফলাফল)।
116	`fprintf(fid, 'GAUSS-SEIDEL LOAD FLOW RESULT\n');`	ফাইলে শিরোনাম।
117	`fprintf(fid, 'Result found at iteration number: %d\n\n', iter);`	কত নম্বর iteration-এ উত্তর পাওয়া গেছে; %d-এ iter বসে। \n\n = দুটি নতুন লাইন।
119	`fprintf(fid, 'BUS ADMITTANCE MATRIX (YBus) in pu\n');`	Ybus-এর শিরোনাম ফাইলে।
120	`for i = 1:no_bus`	প্রতিটি row-এর জন্য loop।
121	`  for k = 1:no_bus`	প্রতিটি column-এর জন্য loop।
122	`  fprintf(fid, '%9.4f %+9.4fj   ', real(Y(i,k)), imag(Y(i,k)));`	প্রতিটি element-এর real ও imaginary অংশ ফাইলে (line 63-এর মতো)।
123	`  end`	column loop শেষ।
124	`  fprintf(fid, '\n');`	row শেষে নতুন লাইন।
125	`  end`	row loop শেষ।
126	`fprintf(fid, '\n');`	Ybus-এর পরে ফাঁকা লাইন।
128	`  fprintf(fid, 'Bus V(pu) Angle(deg) P(pu) Q(pu)\n');`	Bus ফলাফল টেবিলের শিরোনাম।
129	`for i = 1:no_bus`	প্রতিটি bus-এর জন্য loop।
130	`  fprintf(fid, '%3d %8.4f %10.4f %9.4f %9.4f\n', ...`	Format: %3d = bus নম্বর, %8.4f ইত্যাদি = নির্দিষ্ট চওড়ায় দশমিক, যাতে column সোজা থাকে। ... = পরের লাইনে চলবে।
131	`  i, V(i), delta(i)*180/pi, P(i), Q(i));`	যে মানগুলো বসবে: bus নম্বর, V, delta×180/pi (ডিগ্রি), P, Q। এখানে slack-এর P ও Q, PV-এর Q এবং PQ-এর V ও δ — এগুলোই load flow-এর হিসাব করা উত্তর।
132	`  end`	loop শেষ।
134	`  fprintf(fid, '\nLINE FLOWS S(i,j) = P(i,j) + jQ(i,j) (pu)\n');`	Line flow অংশের শিরোনাম।
135	`for i = 1:no_bus`	বাইরের loop — i = 1 থেকে no_bus।
136	`  for k = i+1:no_bus`	ভেতরের loop — k = i+1 থেকে; প্রতিটি bus-জোড়া একবার।

লাইন	কোড	বাংলা ব্যাখ্যা
137	`if Y(i,k) ~= 0`	Y(i,k) ~= 0 মানে i ও k-এর মাঝে line আছে (~= = "সমান নয়")। Line না থাকলে (যেমন 1-4) flow হিসাবের দরকার নেই।
138	`y_ik = -Y(i,k);`	Ybus-এর off-diagonal = −y, তাই আসল line admittance y_ik = −Y(i,k) (যেমন −(−2 + j6) = 2 − j6)।
139	`Sik = Vc(i) * conj( (Vc(i)-Vc(k)) * y_ik );`	Equation (5): Sik = Vi × [(Vi − Vk)·yik]*। (Vi − Vk)·yik = line current Iik, আর S = V·I*। Bus i থেকে k-এর দিকে পাঠানো power।
140	`Ski = Vc(k) * conj( (Vc(k)-Vc(i)) * y_ik );`	উল্টো দিক (k → i)। Line-এ loss থাকায় Sik আর Ski সমান-বিপরীত নয়; দুটো যোগ করলে line loss। তাই দুই দিকই হিসাব করা হয়।
141	`fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', i, k, real(Sik), imag(Sik));`	Sik-এর real (P) ও imaginary (Q) ফাইলে, যেমন "1 -> 2 : -0.0953 + j 0.3120"। ঋণাত্মক P = power আসলে উল্টো দিকে যাচ্ছে।
142	`fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', k, i, real(Ski), imag(Ski));`	Ski ফাইলে লেখা।
143	`end`	if শেষ।
144	`end`	ভেতরের loop শেষ।
145	`end`	বাইরের loop শেষ।
147	`fprintf(fid, '\nITERATION HISTORY (angles in degree)\n');`	Iteration history-এর শিরোনাম।
148	`fprintf(fid, 'Iter');`	প্রথম column-এর নাম "Iter"।
149	`for i = 2:no_bus`	bus 2 থেকে শেষ bus পর্যন্ত column নামের loop।
150	`fprintf(fid, '   delta%d(deg)', i);`	"delta2(deg)", "delta3(deg)" ... লেখা।
151	`end`	loop শেষ।
152	`fprintf(fid, '   max change (del)\n');`	শেষ column-এর নাম ও নতুন লাইন।
153	`for n = 1:iter`	প্রতিটি iteration-এর জন্য loop।
154	`fprintf(fid, '%3d ', n);`	iteration নম্বর লেখা।
155	`for i = 2:no_bus`	প্রতিটি bus (2 থেকে)-এর জন্য loop।
156	`fprintf(fid, '%14.4f', delta_hist(n,i));`	ওই iteration-এ bus i-এর angle (ডিগ্রি) — delta_hist থেকে।
157	`end`	loop শেষ।
158	`fprintf(fid, '%18.6f\n', del_hist(n));`	ওই iteration-এর del (6 দশমিক) ও নতুন লাইন।
159	`end`	iteration loop শেষ।
160	`fclose(fid);`	fclose() ফাইল বন্ধ করে — বন্ধ না করলে লেখা পুরো সংরক্ষণ নাও হতে পারে।
162	`fprintf('\nConverged at iteration %d. Results saved to load_flow_result.txt\n', iter);`	fid ছাড়া fprintf = screen-এ লেখা। User-কে জানায় কত iteration-এ converge করেছে ও ফলাফল কোথায়।
164	`%% Convergence graph`	Section — Graph 1: convergence graph।
165	`figure;`	figure = নতুন graph window, যাতে দুটি graph আলাদা থাকে।

লাইন	কোড	বাংলা ব্যাখ্যা
166	`semilogy(1:iter, del_hist, '-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');`	semilogy() — y-axis log scale। del 0.085 থেকে 0.00011 পর্যন্ত কমে (প্রায় 800 গুণ); সাধারণ axis-এ শেষের মান শূন্যের সাথে মিশে যেত, log scale-এ প্রতিটি ধাপ দেখা যায়। '-o' = রেখা + বৃত্ত, LineWidth = পুরুত্ব, MarkerFaceColor 'b' = নীল ভরা বৃত্ত।
167	`hold on;`	hold on — এরপর যা আঁকা হবে একই graph-এ যোগ হবে।
168	`plot([1 iter], [0.000170 0.000170], 'r--', 'LineWidth', 1.2);`	y = 0.00017-এ অনুভূমিক লাল ড্যাশ রেখা ('r--') — tolerance সীমা। del এর নিচে নামলেই loop থামে।
169	`text(1, 0.000170, '  Tolerance = 0.00017', 'Color', 'r', 'VerticalAlignment', 'bottom');`	text() graph-এর (1, 0.00017) অবস্থানে লাল লেখা; VerticalAlignment bottom = রেখার ঠিক উপরে।
170	`hold off;`	hold off — যোগ করা শেষ।
171	`grid on;`	grid on — গ্রিড লাইন, মান পড়তে সুবিধা।
172	`xlabel('Iteration number');`	x-axis-এর নাম।
173	`ylabel('Maximum change, del');`	y-axis-এর নাম।
174	`legend('del (largest change)', 'Tolerance', 'Location', 'northeast');`	legend() — কোন রেখা del আর কোনটা tolerance; উপরে ডান কোণে।
175	`title('Convergence of Gauss-Seidel Load Flow');`	Graph-এর শিরোনাম।
176	`set(gca, 'XTick', 1:iter);`	gca = বর্তমান axis। x-axis-এ শুধু 1, 2, 3 ... পূর্ণসংখ্যা দাগ, কারণ iteration ভগ্নাংশ হয় না।
178	`%%  Bus voltage angles vs iterations`	Section — Graph 2: iteration-এর সাথে bus angle।
179	`figure;`	নতুন graph window।
180	`plot(1:iter, delta_hist(:,2:no_bus), '-o', 'LineWidth', 1.5);`	plot() — delta_hist(:,2:no_bus) = সব iteration-এ bus 2 থেকে শেষ bus-এর angle। Matrix দিলে plot প্রতিটি column-এর আলাদা রেখা আঁকে — প্রতিটি bus একটি রেখা। Slack বাদ কারণ তার angle সবসময় 0।
181	`grid on;`	গ্রিড চালু।
182	`xlabel('Iteration number');`	x-axis-এর নাম।
183	`ylabel('Angle (degree)');`	y-axis-এর নাম (ডিগ্রি)।
184	`title('Bus voltage angles through the iterations');`	Graph-এর শিরোনাম।
185	`legend(arrayfun(@(k) sprintf('\\delta%d', k), 2:no_bus, 'UniformOutput', false), 'Location', 'best');`	arrayfun() 2:no_bus-এর প্রতিটি k-এর জন্য sprintf চালিয়ে {'\delta2', '\delta3', ...} লেবেল তালিকা বানায় (UniformOutput false = ফলাফল cell)। \delta লিখলে MATLAB গ্রিক δ দেখায়। যত bus-ই হোক legend আপনা থেকে ঠিক হয়।
186	`set(gca, 'XTick', 1:iter);`	x-axis-এ শুধু পূর্ণসংখ্যা iteration নম্বর।

এই অংশের function: কী করে ও কেন

Function	কী করে	কেন ব্যবহার (লজিক)
`fopen(), fclose()`	ফাইল খোলা ও বন্ধ	ফলাফল text file-এ রাখা (manual-এর নির্দেশ); বন্ধ না করলে সংরক্ষণ নাও হতে পারে।

Function	কী করে	কেন ব্যবহার (লজিক)
`fprintf(fid, …)`	ফাইলে লেখা	fid দিলে লেখা screen-এ না গিয়ে ফাইলে যায়; format দিয়ে টেবিল সোজা।
`~=`	সমান নয়	Y(i,k) ~= 0 → line আছে, তবেই flow হিসাব।
`figure`	নতুন graph window	দুটি graph আলাদা।
`semilogy()`	y-axis log scale	del 800 গুণ কমে; log scale-এ সব ধাপ দেখা যায়।
`plot()`	রেখা graph	Tolerance রেখা; matrix দিলে প্রতিটি bus-এর আলাদা রেখা।
`hold on/off`	একই graph-এ একাধিক রেখা	del ও tolerance একসাথে।
`text(), legend()`	লেখা বসানো; রেখার পরিচয়	Tolerance-এর মান লেখা; কোন রেখা কী তা বোঝানো।
`grid, xlabel, ylabel, title`	গ্রিড ও নাম	Graph পড়তে সুবিধা।
`arrayfun()`	তালিকার প্রতিটি উপাদানে function	যত bus-ই হোক δ2, δ3 … legend আপনা থেকে।
`set(gca,'XTick',…)`	axis-এর দাগ ঠিক করা	Iteration পূর্ণসংখ্যা, তাই শুধু 1, 2, 3 …

লজিক: "কেন এভাবে?"

- Line flow দুই দিক থেকে কেন? — line-এ loss থাকায় পাঠানো আর পৌঁছানো power আলাদা; Sik + Ski = line loss।
- y_ik = −Y(i,k) কেন? — Ybus-এর off-diagonal-এ −y রাখা হয়েছিল; আসল admittance পেতে আবার −।
- Sik = Vi·[(Vi − Vk)yik]* — (Vi − Vk)yik হলো line current, আর S = V·I*।
- ঋণাত্মক P মানে? — power উল্টো দিকে যাচ্ছে; যেমন 1→2 = −0.0953, কারণ bus 2-এর angle (1.65°) বেশি, real power বেশি angle থেকে কম angle-এ যায়।
- Log scale কেন? — del 0.085 থেকে 0.00011; সাধারণ scale-এ শেষের ধাপ শূন্যের সাথে মিশে যেত।
- Angle graph-এ slack নেই কেন? — slack-এর angle সবসময় 0 (reference)।

সম্ভাব্য ভাইভা প্রশ্ন

প্রশ্ন 1: Line loss কীভাবে বের করবে?

উত্তর: একই line-এর দুই দিকের flow যোগ: Sik + Ski। মোট loss = 0.0427 + j0.1280 pu।

প্রশ্ন 2: সবচেয়ে বেশি loaded line কোনটি?

উত্তর: Line 3-4, প্রায় 0.447 pu (bus 4 থেকে bus 3-এর দিকে)।

প্রশ্ন 3: Convergence graph থেকে কী বোঝা যায়?

উত্তর: del প্রায় সরলরেখায় (log scale) কমে — Gauss-Seidel-এর linear convergence; 10তম iteration-এ tolerance-এর নিচে।

প্রশ্ন 4: Power balance মেলে কি?

উত্তর: Q: 0.2558 − 0.5278 + 0.5 − 0.1 = 0.1280 = Q loss ✓। P-তে 0.0014 pu পার্থক্য, কারণ tolerance 0.00017-এ থামা হয়েছে।

শেষে: পুরো program এক নজরে

ধাপ	কী হয়	কে বলবে
১	Bus সংখ্যা ও bus ডেটা (inf = অজানা)	Member 1
২	Flat start, complex voltage, safety check, PV চেনা	Member 1
৩	Line ডেটা → Ybus, বিচ্ছিন্ন bus পরীক্ষা, Ybus দেখানো	Member 2
৪	Iteration counter ও history array	Member 2
৫	Gauss-Seidel loop (del < 0.00017 পর্যন্ত)	Member 3
৬	Slack bus P1, Q1	Member 3
৭	Text file: Ybus, bus ফল, line flow, iteration history	Member 4
৮	Convergence ও angle graph	Member 4